Universidad Mariano Gálvez de Guatemala

Facultad de Ingeniería en Sistemas

Proyecto del curso

ER-AFD

Convertidor de expresiones regulares a autómatas finitos deterministas

Documento 03
Casos de prueba

Integrantes

Carlos Abraham Méndez Asencio - Carné 94902417370

Maycol Alexander Ortiz Flores - Carné 9490-24-27213

Resultados de la verificación en Ubuntu

El conjunto de pruebas del proyecto ER-AFD se ejecutó localmente en Ubuntu Desktop 26.04.1 LTS de 64 bits. Se evaluaron las operaciones del lenguaje, las tablas de transiciones, la simulación de cadenas y la extensión de tokenización. Las pruebas están diseñadas para poder repetirse sin utilizar un motor de expresiones regulares como referencia de aceptación.

Indicador	Resultado documentado
Pruebas unittest	39 ejecutadas; 0 fallos y 0 errores.
Casos con resultado esperado	1,499 correctos de 1,499.
Matriz de confusión	TP 710 · TN 789 · FP 0 · FN 0.
Exactitud, precisión, sensibilidad	100 % en la muestra evaluada.
Comparaciones AFND / AFD	20,306 coincidencias de 20,306.
Verificador de integración	16 comprobaciones con retorno esperado.

Entorno y procedimiento

Las evidencias registran Ubuntu Desktop 26.04.1 LTS x86_64, kernel 7.0.0-42-generic, Python 3.14.4 y GNU Make 4.4.1. Los comandos se ejecutaron con el usuario normal erafd, UID 1000, sin necesitar root. La fecha registrada es el 2 de noviembre de 2026.

```
cd ~/er-afd
bash tools/verify_linux.sh
```

Cada ejecución genera una carpeta nueva en doc/evidencias/linux/. El resultado satisfactorio se reconoce por el mensaje VERIFICACION LINUX COMPLETADA y por completed: true en resultado.json. El historial anterior de Windows se conserva en otro archivo de métricas.

El 100 % observado no significa que todas las expresiones regulares y textos posibles hayan sido comprobados. Describe únicamente los resultados obtenidos con los casos de prueba incluidos en el informe.

Diseño y cobertura de los casos

La batería de pruebas está organizada para revisar las operaciones del lenguaje y las reglas de construcción del autómata. Se incluyen entradas válidas, inválidas, clases de caracteres y situaciones de integración con el sistema operativo.

Área	Aspectos que se comprueban
Lenguaje	Concatenación, unión, precedencia, Kleene, grupos y coincidencia completa.
Clases	Conjuntos positivos y negativos, rangos, macros, los 128 caracteres ASCII y solapamientos.
Escapes	Operadores literales, diagonal, espacios, h00, h0A, h20 y h7F.
Épsilon	Cadena vacía, concatenaciones, alternativas y ciclos sin bloqueo.
Errores	Rangos invertidos, sintaxis incompleta, no ASCII y límites de recursos.

Área	Aspectos que se comprueban
Estructura	Estados iniciales q0/Q0, finales por subconjuntos, destinos declarados y matriz completa.
Integración	Códigos 1/0/2, JSON, DOT, archivos, Make y reconstrucción.
Tokenizador	Coincidencia más larga, prioridad, último final válido, skip y posiciones.

Resultados esperados y comparación diferencial

Los resultados de los casos de lenguaje se definieron por separado de la implementación. Las pruebas de conjuntos emplean predicados de pertenencia directos. A su vez, el simulador de pruebas del AFND calcula cierres y avances por conjuntos sin consultar la matriz del AFD.

La comparación diferencial evalúa 781 cadenas de longitudes de cero a cuatro, formadas con el alfabeto {a, b, c, 1, espacio}, frente a 26 expresiones regulares. El total es de 20,306 comparaciones. Ambos simuladores comparten el AFND producido por Thompson, de modo que esta comparación verifica principalmente la construcción de subconjuntos.

La exactitud se calcula como (TP + TN) / N; la precisión, como TP / (TP + FP); y la sensibilidad, como TP / (TP + FN). Los casos adicionales del ejecutable Unix y del escape de espacios no se agregaron a las cifras de 1,499 y 20,306.

Casos verificables

Las siguientes entradas representan distintas operaciones y resultados. Los códigos corresponden a la interfaz er-afd: 1 para aceptación, 0 para rechazo y 2 para error.

Expresión regular	Cadena de entrada	Resultado esperado
(a|b)*abb	aabb	Aceptada / 1
(a|b)*abb	aab	Rechazada / 0
a*	Vacía	Aceptada / 1
ε	Vacía	Aceptada / 1
aεb	ab	Aceptada / 1
[a-c][0-2]	b1	Aceptada / 1
[^abc]	z	Aceptada / 1
a|.	a y b	Ambas aceptadas / 1
\A\a*	Guatemala	Aceptada / 1
\N\N*	12a	Rechazada / 0
\h20\h41	Espacio seguido de A	Aceptada / 1
\n	LF	Rechazada / 0
[z-a]	a	Error / 2
\h80	Cualquier entrada	Error / 2

La verificación distingue los rechazos legítimos de los errores. Por ejemplo, el rango [z-a] es sintácticamente inválido y devuelve 2; no debe aparecer como una cadena simplemente rechazada.

Evidencias de ejecución en Linux

Las seis capturas se registraron durante la ejecución en Ubuntu dentro de doc/evidencias/linux/20261102T180611Z/. Los originales se encuentran en el paquete de fuentes y en el PDF de casos de prueba proporcionado. Se describen aquí junto con los comandos y resultados documentados, sin crear capturas de terminal artificiales.

Entorno de Ubuntu

La captura 01_entorno.png muestra la distribución, arquitectura x86_64, versiones de Python y Make, UID del usuario y zona horaria America/Guatemala. Con ello se identifica la máquina utilizada para la ejecución local.

```
python3 --version
make --version
uname -m
id
```

Suite y métricas

La captura 02_suite_y_metricas.png conserva la salida de make test. El registro confirma 39 pruebas terminadas en OK, 1,499 casos con resultado esperado y 20,306 comparaciones del AFND frente al AFD, además del resumen del verificador de integración.

```
make test
```

Tablas y aceptación

La captura 03_automatas_aceptacion.png corresponde a una evaluación aceptada y muestra los estados, matrices, transiciones y recorrido. El ejemplo ejecuta ./dist/er-afd a a y termina con el mensaje Cadena aceptada y retorno 1.

```
./dist/er-afd a a
```

Rechazo, error y cadena vacía

En 04_rechazo_y_error.png se documentan tres situaciones distintas: la expresión a frente a b devuelve 0; el rango [z-a] produce error 2; y ε frente a una cadena vacía devuelve aceptación 1. Son casos importantes para no confundir errores de entrada con rechazos válidos.

```
./dist/er-afd a b
./dist/er-afd '[z-a]' a
./dist/er-afd 'ε' ''
```

Tokenización y escapes

La captura 05_lexer_y_escapes.png presenta la salida del tokenizador con siete tokens y sus posiciones, además de una comprobación de un espacio escapado en Bash. El caso hola\ mundo frente a hola mundo se registra como aceptado.

Limpieza y reconstrucción

La captura 06_limpieza_y_reconstruccion.png muestra que make clean no borra fuentes, PDF ni evidencias. Después, make build reconstruye los ejecutables y se comprueba la aceptación de aaab para la expresión a*b.

```
make clean
make build
./dist/er-afd "a*b" "aaab"
```

Integridad y repetición de resultados

La carpeta de evidencias incluye transcripcion.log, retornos.tsv, entorno.json, resultado.json y metricas_linux.json. codigo_ejecutado_sha256.json identifica la versión de los archivos fuente utilizada y SHA256_EVIDENCIAS.json permite contrastar los archivos guardados. Estos datos se deben conservar junto con las capturas originales.

Archivo o carpeta	Contenido
transcripcion.log	Comandos y resultados de la sesión de comprobación.
retornos.tsv	Códigos de retorno de cada proceso ejecutado.
entorno.json	Versión y características del sistema operativo y las herramientas.
resultado.json	Resumen de los pasos del verificador y campo completed.
metricas_linux.json	Resultados cuantitativos de las pruebas ejecutadas en Ubuntu.
codigo_ejecutado_sha256.json	Identificación SHA-256 de los fuentes sometidos a prueba.
SHA256_EVIDENCIAS.json	Huellas para comprobar la integridad de los archivos registrados.

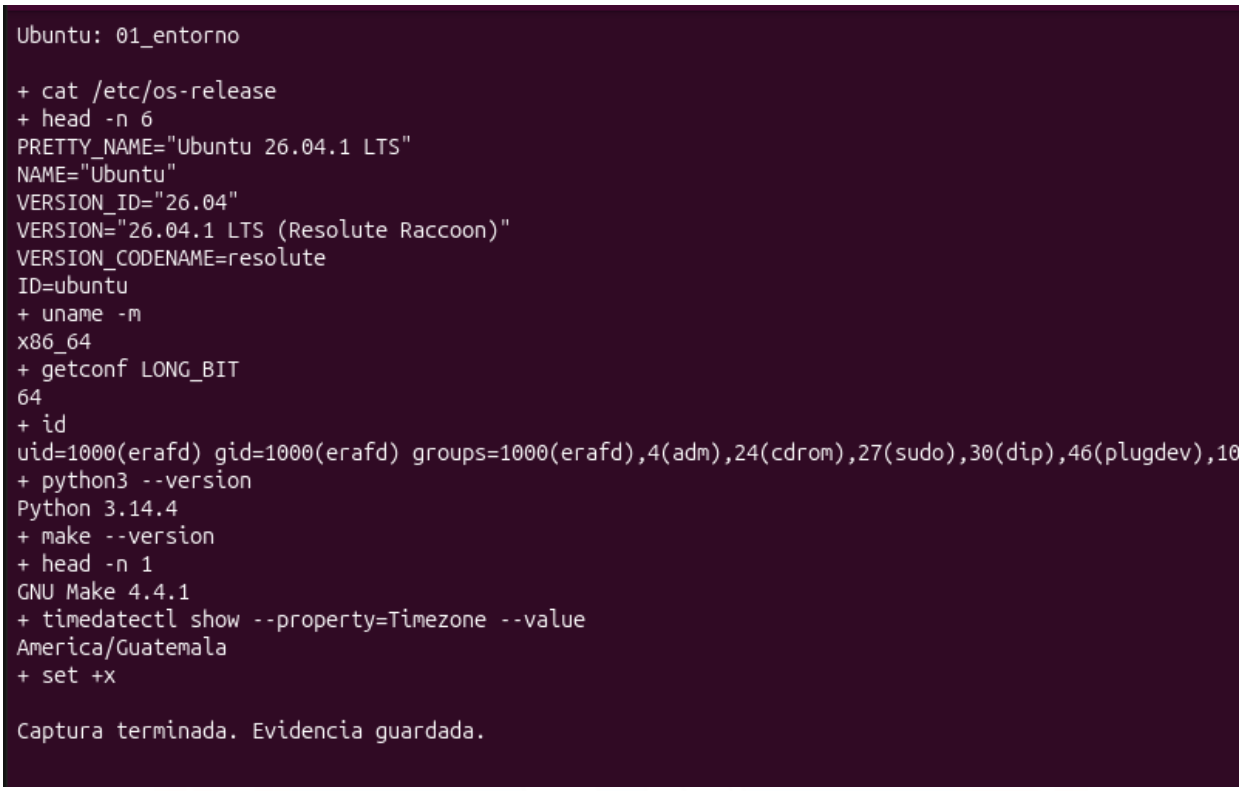
El conjunto documentado contiene resultados locales de Ubuntu. Una ejecución de GitHub Actions requiere publicar el repositorio y ejecutar su workflow; se mantiene como verificación independiente y no se presenta como resultado de CI ya obtenido.

Reproducibilidad

Para repetir las pruebas se parte de una copia completa del proyecto, se accede a su directorio principal y se ejecuta bash tools/verify_linux.sh. Si el procedimiento termina satisfactoriamente, debe conservarse la nueva carpeta de resultados sin sobrescribir las capturas anteriores.

Anexo: Entorno de Ubuntu

Se identifican Ubuntu 26.04.1 LTS, la arquitectura x86_64, Python 3.14.4, GNU Make 4.4.1 y el usuario erafd con UID 1000.



Detalle ampliado de la terminal de Ubuntu. El encuadre facilita la lectura; el PNG original completo se conserva sin modificaciones.

Original: 01_entorno.png.

Anexo: Suite y métricas

La ejecución de make test termina con 39 pruebas y OK. El resumen confirma 16 comprobaciones, 1499 casos con resultado esperado y 20306 comparaciones AFND/AFD.



Detalle ampliado de la terminal de Ubuntu. El encuadre facilita la lectura; el PNG original completo se conserva sin modificaciones.

Original: 02_suite_y_metricas.png.

Anexo: Tablas y aceptación

El ejecutable ./dist/er-afd evalúa a frente a a. Se muestran el AFND, el AFD, sus estados y transiciones, el recorrido y la aceptación con retorno 1.

```
Ubuntu: 03_automatas_aceptacion

+ ./dist/er-afd a a
AFND
Q={q0,q1}
Σ={a,ε}
Inicial=q0
F={q1}
q0|a|{q1}

AFD
Q={Q0,Q1,Q2}
Σ={a}
Γ={a,C1}
Inicial=Q0
F={Q1}
Γ: particiones disjuntas de ASCII; incluyen la clase residual.
a={a}
C1={\h00..`,b..\h7F}
Subconjuntos del AFND:
Q0={q0}
Q1={q1}
Q2={}
Q0|a|Q1
Q0|C1|Q2
Q1|a|Q2
Q1|C1|Q2
Q2|a|Q2
Q2|C1|Q2

Recorrido
Q0|a|Q1
Cadena aceptada
+ estado=1
+ printf 'Retorno: %s (1 = aceptada)\n' 1
Retorno: 1 (1 = aceptada)
+ set +x

Captura terminada. Evidencia guardada.
```

Detalle ampliado de la terminal de Ubuntu. El encuadre facilita la lectura; el PNG original completo se conserva sin modificaciones.

Original: 03_automatas_aceptacion.png.

Anexo: Rechazo, error y cadena vacía

a frente a b produce rechazo con retorno 0; [z-a] genera error 2; la expresión épsilon frente a la cadena vacía acepta con retorno 1.



Detalle ampliado de la terminal de Ubuntu. El encuadre facilita la lectura; el PNG original completo se conserva sin modificaciones.

Original: 04_rechazo_y_error.png.

Anexo: Tokenización y escapes

El lexer genera siete tokens con sus posiciones. La expresión hola\ mundo frente a hola mundo confirma el espacio escapado en Bash y la aceptación con retorno 1.

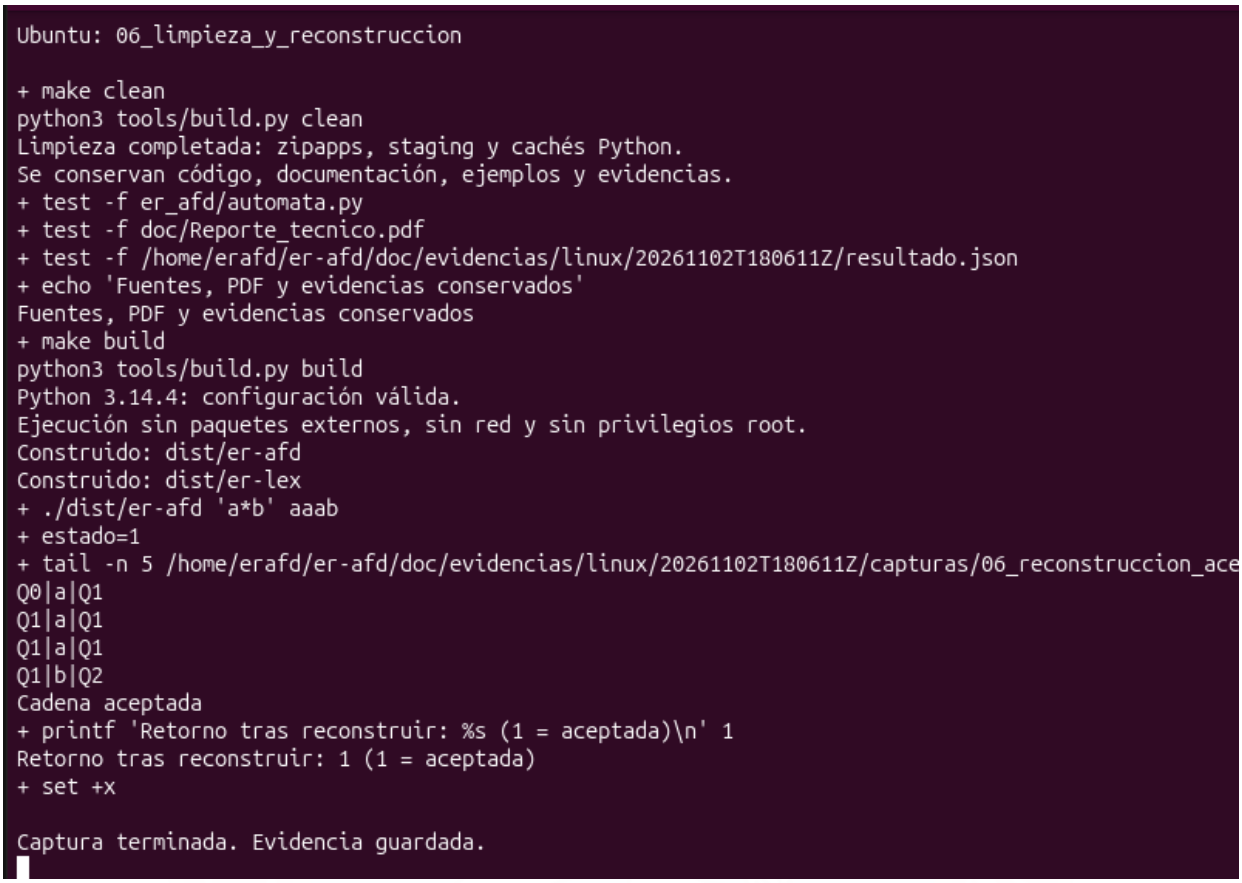


Detalle ampliado de la terminal de Ubuntu. El encuadre facilita la lectura; el PNG original completo se conserva sin modificaciones.

Original: 05_lexer_y_escapes.png.

Anexo: Limpieza y reconstrucción

make clean conserva fuentes, documentos y evidencia. make build reconstruye los ejecutables y a*b frente a aaab vuelve a aceptar con retorno 1.



Detalle ampliado de la terminal de Ubuntu. El encuadre facilita la lectura; el PNG original completo se conserva sin modificaciones.

Original: 06_limpieza_y_reconstruccion.png.

Anexo: Validación adicional en Ubuntu

Se añadió una batería independiente para ampliar los casos sin reemplazar las métricas históricas. Se ejecutó con Python 3.14.4 en Linux x86_64, mediante un usuario regular, y terminó con passed: true.

Comprobación adicional	Resultado
Casos de lenguaje	800 correctos de 800
Clasificación	TP 222 · TN 578 · FP 0 · FN 0
Patrones inválidos	9 de 9 detectados
Procesos de consola	25 de 25 con JSON y retorno esperado
Métricas históricas	SHA-256 idénticos antes y después de las pruebas

Cómo se formaron los casos nuevos

Cuatro patrones se contrastaron con los 128 caracteres ASCII, el vacío y una cadena de dos caracteres: controles, complemento de controles, lenguaje vacío y su cerradura. Son 520 casos. Otros 259 enumeran cadenas de longitud 0 a 3 sobre {a,b,0,9,_,tabulador} para un conjunto negativo compuesto. Se agregaron 21 ejemplos dirigidos; el total es 800 pares ER/entrada distintos del conjunto histórico.

Los valores esperados proceden de pertenencia, límites numéricos de ASCII, longitud e igualdad de texto o bytes; no de otro motor regex. La API se comprobó en el AFD, el AFND y la simulación con recorrido.

Separación de resultados

El histórico permanece en 39 pruebas unitarias, 1499 casos con oráculo y 20306 comparaciones AFND/AFD. La batería adicional se informa por separado. Los nueve patrones inválidos también se comprueban en la CLI; no deben sumarse como nueve errores distintos de esos procesos.

Repetición

Desde la raíz del proyecto: python3 -m tests.additional_validation --output "$HOME/erafd-adicionales-nueva.json". El archivo debe ser nuevo; se requiere Linux de 64 bits y usuario sin root.

Anexo: Casos nuevos representativos

Expresión o entrada	Criterio y resultado esperado
[^\h00-\h7F]	Complemento del ASCII completo: rechaza todos sus caracteres y el vacío.
([^\h00-\h7F])*	Solo acepta el vacío; distingue lenguaje vacío de épsilon.
[^0-9][^0-9]*	Acepta abc_! y rechaza A7: texto no vacío sin dígitos.
[^\N\A\a_]	Acepta tabulador y guion; rechaza guion bajo y dígitos.
\A\A\h2D\N\N\N	Acepta GT-007; rechaza gt-007, GT-07 y GT-00A.
\t	Acepta la letra t y rechaza un tabulador: escape genérico.
\h00\h09\h0D\h0A\h7F	Acepta bytes 00 09 0D 0A 7F; rechaza si falta CR.
-- -a -a	Acepta el texto literal -a al terminar las opciones con --.
Archivo con byte FF o 80	Error 2: son bytes fuera de ASCII, no rechazos del lenguaje.

Errores de expresión y límites

Se verificaron escapes hexadecimales incompletos o fuera de ASCII, rangos invertidos, macros empleadas como extremos de rango, Unicode dentro de un conjunto y una alternativa sin operando. En la consola se exigió retorno 2, JSON con error y ausencia de traceback. Otro caso comprueba el límite de estados del AFD.

Los controles NUL, TAB, CR, LF y DEL se transportaron en archivos binarios para conservar exactamente sus bytes. NUL no puede pasarse como argumento de un proceso. Los registros de la CLI conservan las entradas hexadecimales y stdout/stderr.

Evidencia adicional: doc/evidencias/linux/adicionales_20261103T033444Z/resultado_adicional.json. El ejecutor tests/additional_validation.py y el resumen doc/PRUEBAS_ADICIONALES.md permiten reproducir la revisión.

Los resultados amplían la muestra documentada. No equivalen a probar todas las expresiones ni a garantizar precisión universal.